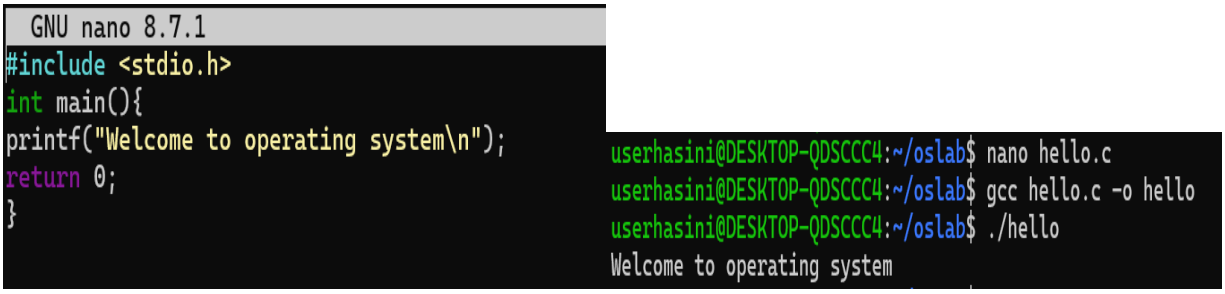
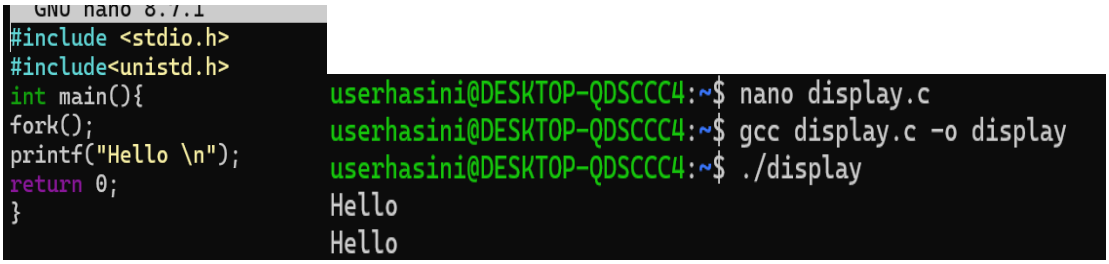
WEEK-1:

Program 1:

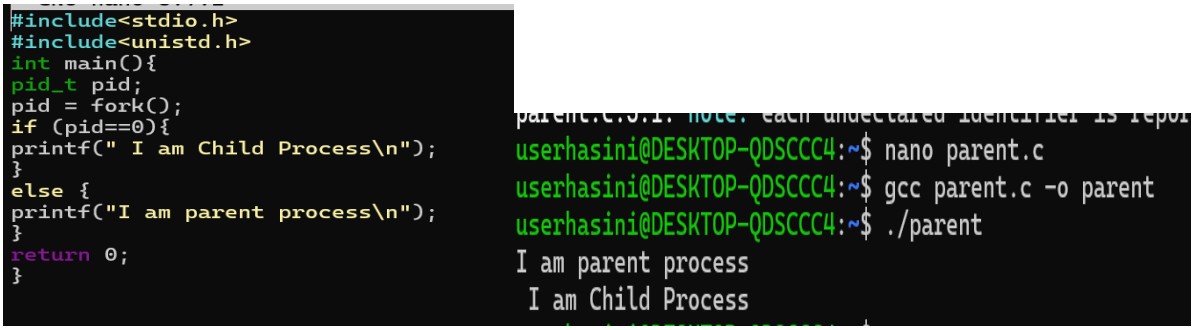


Program 2:

Program 3:



Program 4:



Program 5:

```c
#include<stdio.h>
#include<unistd.h>
int main(){
pid_t pid;
pid = fork();
if (pid==0){
printf("Child PID=%d\n",getpid());
printf("Parent PID=%d\n",getpid());
}
else {
printf("Parent PID = %d\n",getpid());
}
return 0;
}
```

```
userhasini@DESKTOP-QDSCCC4:~$ gcc parent.c -o parent
userhasini@DESKTOP-QDSCCC4:~$ ./parent
Parent PID = 795
Child PID=796
Parent PID=796
```

```c
  GNU nano 8.7.1
#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>
int main(){
pid_t pid;
pid=fork();
if (pid==0){
printf("child running");
}
else {
wait(NULL);
printf("Parent Running After Child\n");
}
return 0;
}
```

```
userhasini@DESKTOP-QDSCCC4:~$ gcc wait.c -o wait
userhasini@DESKTOP-QDSCCC4:~$ ./wait
child runningParent Running After Child
```

Program 7:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(){
char cmd[100];
printf("Enter linux command:");
scanf("%s", cmd);
pid_t pid = fork();
if (pid==0){
printf("child pid: %d\n",getpid());
execlp(cmd, cmd, NULL);
perror("Exec Failed");
}
else if(pid > 0){
printf("Parent PID: %d\n", getpid());
wait(NULL);
printf("Child process completed.\n");
}
else
{
printf("Fork Failed\n");
}
return 0;
}
```

```
userhasini@DESKTOP-QDSCCC4:~$ nano child.c
userhasini@DESKTOP-QDSCCC4:~$ gcc child.c -o child
userhasini@DESKTOP-QDSCCC4:~$ ./child
Enter linux command:ls
Parent PID: 649
child pid: 650
child    display    file1.txt  hello.c  parent    practical1.c       re
child.c  display.c  hello      oslab    parent.c  practical1.c.save  re
Child process completed.
userhasini@DESKTOP-QDSCCC4:~$
```

Program 8:

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main(){
int src, dest;
char buffer[1024];
int bytes;
src = open("input.txt", O_RDONLY);
if(src < 0){
printf("Cannot open source file\n");
return 1;
}
dest = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
while((bytes = read(src, buffer, sizeof(buffer))) > 0)
{
write(dest, buffer, bytes);
}
close(src);
close(dest);
printf("File copied successfully.\n");
return 0;
}
```